

BACKEND & RUNTIME MECHANICS

Asynchronous JavaScript

Callbacks, Promises, and Async/Await pipelines

The Event Loop

Understanding single-threaded concurrent compilation

Why Asynchronous?

JavaScript executes code synchronously by default, running on a single main thread. Long-running computational processes or network requests block the interface entirely if not handled natively.

- ❑ Non-Blocking: Offloads time-consuming tasks to the browser runtime infrastructure.
- ❑ Network Operations: Fetching server API payloads data streams gracefully.
- ❑ File I/O: Handling operations smoothly without freezing interactive modules.

600 × 400

Callbacks

Functional Foundations

A callback is a function passed into another function as an argument, scheduled to run once an outer operation completes execution.

Callback Hell

Deeply nested callbacks create unmaintainable, pyramid-shaped structures. This pattern complicates clean error handling and tracking stack frames.

Promises

A structured container for future computed values

Promise Anatomy



Pending

Initial operating status. The underlying asynchronous calculation is still working towards resolution.



Fulfilled

The action completed successfully. Calls the attached `.then()` callback containing your result payload.



Rejected

The operation failed completely. Hands processing off to the `.catch()` block for remediation errors.

Promise Chaining

- ❑ Linear Pipelines: Flatten nested structures by returning new promises from inside a `.then()` callback.
- ❑ Centralized Errors: Catch errors across any upstream segment using a single, unified `.catch()` block at the bottom.
- ❑ Clean Transitions: Keep asynchronous logic organized sequentially without breaking readability constraints.

Modern Async / Await

Syntactic sugar wrapped around classical Promise behaviors

The Async Paradigm

Declaring a function with the `async` keyword guarantees it returns a promise automatically, wrapping arbitrary returned primitives immediately.

The `await` operator pauses code execution inline until the targeted promise settles, simplifying data assignments across synchronous steps.

600 × 400

Error Control Flow

Block Element	Functional Responsibilities	Downstream Strategy
<code>try</code>	Isolates normal sequential logic containing active <code>await</code> boundaries.	Executes straight through to completion if operations succeed.
<code>catch</code>	Intercepts any rejections or unexpected exceptions automatically.	Recovers data status or sends diagnostic logs back to standard outputs.
<code>finally</code>	Executes consistently regardless of the operational path outcome.	Tears down database links, updates UI flags, or clears loading states.

Execution Comparison

Promise Chaining Syntax

Uses explicit, callback-driven pipelines to map incoming data items.

```
fetch('/api/v1/users')  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```

Async / Await Alternative

Flattens execution into clean, procedural steps that mimic standard synchronous structures.

```
try {  
  const res = await fetch('/api/v1/users');  
  const data = await res.json();  
  console.log(data);  
} catch (err) {  
  console.error(err);  
}
```


Questions?

Architecting production-ready asynchronous pipelines



www.zindua.io | joseph.kwanusu@zindua.io